



Ejercicio 1

```
def lee_estudios(ruta_fichero: str) -> list[Estudio]:
    res = []
    with open(ruta_fichero, encoding="utf-8") as f:
        lector = csv.reader(f)
        next(lector)
        for pais, fecha, enfermedad, categoria, ratios, edad, genero, tratamiento, \
            coste, disponibilidad in lector:
            fecha = parsea_fecha(fecha)
            ratio_mortalidad, ratio_recuperacion = parsea_ratios(ratios)
            edad = parsea_edad(edad)
            genero = parsea_genero(genero)
            coste=float(coste)
            disponibilidad = parsea_disponibilidad(disponibilidad)
            res.append(Estudio(pais,fecha,enfermedad,categoria,ratio_mortalidad, \
                ratio_recuperacion,edad,genero,tratamiento,coste,disponibilidad))
    return res

def parsea_fecha(fecha_str: str) -> date:
    return datetime.strptime(fecha_str, " %d/%m/%Y").date()

def parsea_ratios(ratios_str: str) -> tuple[float, float]:
    # 9.99 ; 85.21
    mortalidad, recuperacion = ratios_str.split(";")
    return float(mortalidad), float(recuperacion) # se puede usar strip() o no

def parsea_edad(edad_str: str) -> tuple[int, int|None]:
    trozos = edad_str.split("-")
    res = None
    if len(trozos) == 1: #No tiene rango superior
        edad = trozos[0].strip().replace("+", "")
        res= Rango(int(edad), None)
    elif len(trozos) == 2: #Tiene rango superior
        res = Rango(int(trozos[0].strip()), int(trozos[1].strip()))
    return res

def parsea_edad2(cadena: str) -> tuple[int, int|None]:
    #0 - 18 # 61+
    if "+" in cadena: #No tiene rango superior
        lim_inferior = cadena[:cadena.index("+")]
        lim_superior = None
    elif "-" in cadena: #Tiene rango superior
        lim_inferior, lim_superior = cadena.split("-")
        lim_superior = int(lim_superior)
    return Rango(int(lim_inferior), lim_superior)
```



```
def parsea_genero(genero_str: str) -> list[str]:
    #Con chequeo estricto
    genero_min = genero_str.strip().lower()
    res = None
    if genero_min == "masculino" or genero_min == "femenino":
        res = {genero_str}
    elif genero_min == "ambos":
        res = {"Masculino", "Femenino"}
    return res

def parsea_genero2(genero_str: str) -> list[str]:
    #Con chequeo más laxo
    generos = {'Femenino', 'Masculino'} if genero_str.lower() == "ambos" \
        else {genero_str}
    return generos

def parsea_disponibilidad (disponib_str: str) -> bool:
    d = disponib_str.strip().lower()
    res = None
    if d == "s":
        res = True
    elif d == "n":
        res = False
    return res
```

Test

```
def mostrar_iterable(iterable: Iterable) -> None:
    for it in iterable:
        print(f"\t{it}")

def test_lee_estudios(estudios: list[Estudio]) -> None:
    print(f"Número de registros leídos: {len(estudios)}")
    print("Los dos primeros registros son:")
    mostrar_iterable(estudios[:2])
    print("Los dos últimos registros son:")
    mostrar_iterable(estudios[-2:])

def main():
    estudios = lee_estudios(r"data\Salud.csv")
    test_lee_estudios(estudios)

if __name__=="__main__":
    main()
```



Ejercicio 2

```
def estudios_de_edad_superan_coste_efectivo_Q3(
    estudios: list[Estudio],
    pais: str,
    rango_edad: Rango) -> list[tuple[date, str, float]]:
    p75 = calcular_percentil(estudios, rango_edad)
    res = []
    for e in estudios:
        if e.pais == pais and esta_en_rango(e, rango_edad) and e.coste_medio >= p75:
            res.append((e.fecha_estudio, e.enfermedad, e.coste_medio))
    return res

def calcular_percentil(estudios: list[Estudio], rango_edad: tuple[int, int|None]) -> float:
    costes = sorted(e.coste_medio for e in estudios
                    if esta_en_rango(e, rango_edad))
    ind_perc = (len(costes) * 3) // 4 ## También ind_perc = int(0.75 * len(costes))
    return costes[ind_perc]

def esta_en_rango(e: Estudio, rango2: Rango):
    rango1 = e.rango_edad
    return rango2.limite_inferior <= rango1.limite_inferior and (
        rango2.limite_superior is None or
        (rango1.limite_superior is not None and rango1.limite_superior <= rango2.limite_superior)
    )

def esta_en_rango2(e: Estudio, rango: Rango) -> bool:
    res = e.rango_edad.limite_inferior >= rango.limite_inferior
    if rango.limite_superior != None and e.rango_edad.limite_superior != None:
        res = res and e.rango_edad <= rango.limite_superior
    elif e.rango_edad.limite_superior == None:
        res = res and rango.limite_superior == None
    return res

def esta_en_rango3(e: Estudio, rango2: Rango):
    rango1 = e.rango_edad
    if rango1.limite_superior != None and rango2.limite_superior != None:
        return rango1.limite_inferior in range(rango2.limite_inferior, rango2.limite_superior+1)\
            and rango1.limite_superior in range(rango2.limite_inferior, rango2.limite_superior+1)
    elif rango1.limite_superior is None and rango2.limite_superior != None:
        return False
    elif rango1.limite_superior != None and rango2.limite_superior is None:
        return rango2.limite_inferior <= rango1.limite_inferior
```

Test

```
def test_estudios_de_edad_superan_coste_efectivo_Q3(
```



```
    estudios: list[Estudio],
    pais: str,
    rango_edad: tuple[int, int|None]) -> list[tuple[date, str, float]]:
print(f"Los estudios del pais {pais} en el rango {rango_edad} que superan Q3 son")
res = estudios_de_edad_superan_coste_efectivo_Q3(estudios, pais, rango_edad)
mostrar_iterable(res)

def main():
    estudios = lee_estudios(r"data\Salud.csv")
    pais, rango = "Reino Unido", Rango(18, 35)
    test_estudios_de_edad_superan_coste_efectivo_Q3(estudios, pais, rango)
    pais, rango = "Argentina", Rango(36, None)
    test_estudios_de_edad_superan_coste_efectivo_Q3(estudios, pais, rango)

if __name__=="__main__":
    main()
```

Ejercicio 3

```
def mayor_periodo_sin_estudios_de_genero(estudios: list[Estudio], generos: set[str]) -> int:
    ord = sorted(e.fecha_estudio for e in estudios \
                 if set(e.genero) <= generos) #también con issubset
    gen_periodos = ((f2 - f1).days for f1, f2 in zip(ord, ord[1:]))
    return max(gen_periodos)
```

#Solución calculando el máximo con bucles

```
def mayor_periodo_sin_estudios_de_genero(estudios: list[Estudio], generos: set[str]):
    estudios_ordenados = sorted((e for e in estudios if e.genero.issubset(generos)),
                                key = lambda x:x.fecha_estudio)
                                #también se puede poner if e.genero <= generos

    maximo = None
    for e1, e2 in zip(estudios_ordenados, estudios_ordenados[1:]):
        dias = (e2.fecha_estudio - e1.fecha_estudio).days
        if maximo is None or maximo < dias:
            maximo = dias
    return maximo
```

#Solución con range

```
def mayor_periodo_sin_estudios_de_genero(estudios: list[Estudio], generos: set[str]) -> int:
    ord = sorted(e.fecha_estudio for e in estudios \
                 if set(e.genero) <= generos) #también con issubset
    gen_periodos = ((ord[i+1] - ord[i]).days for i in range(len(ord) - 1))
    return max(gen_periodos)
```



Test

```
def test_mayor_periodo_sin_estudios_de_genero(estudios: list[Estudio], generos: set[str]) -> int:
    res = mayor_periodo_sin_estudios_de_genero(estudios, generos)
    print(f"El mayor período sin estudios para el género {generos} es: {res}")

def main():
    estudios = lee_estudios(r"data\Salud.csv")
    test_mayor_periodo_sin_estudios_de_genero(estudios, {"Femenino"})
    test_mayor_periodo_sin_estudios_de_genero(estudios, {"Masculino"})
    test_mayor_periodo_sin_estudios_de_genero(estudios, {"Masculino", "Femenino"})

if __name__ == "__main__":
    main()
```

Ejercicio 4

```
def top_enfermedades_mejor_recuperacion_media(estudios: list[Estudio], umbral: float, \
    n: int | None = None) -> list[tuple[str, float]]:
    coste_enfermedades = agrupar_ratios_por_enfermedad(estudios, umbral)
    # También
    # ranking = {enfermedad: statistics.mean(ratios_recuperacion) \
    #             for enfermedad, ratios_recuperacion in coste_enfermedades.items()}
    ranking = {enfermedad: sum(ratios_recuperacion) / len(ratios_recuperacion) \
                for enfermedad, ratios_recuperacion in coste_enfermedades.items()}
    return sorted(ranking.items(), key = lambda x:x[1], reverse=True)[:n]

def agrupar_ratios_por_enfermedad(estudios: list[Estudio], umbral: float) -> dict[str, float]:
    coste_enfermedades = defaultdict(list)
    for e in estudios:
        if e.ratio_mortalidad >= umbral and e.disponibilidad_tratamiento:
            coste_enfermedades[e.enfermedad].append(e.ratio_recuperacion)
    return coste_enfermedades
```

Test

```
def test_top_enfermedades_mejor_recuperacion_media(estudios: list[Estudio], umbral: float, \
    n: int | None = None) -> None:
    top = top_enfermedades_mejor_recuperacion_media(estudios, umbral, n)
    print(f"El top {n} de enfermedades con mejor recuperación media con una ratio de mortalidad \
    igual o superior a {umbral} es: ")
    mostrar_iterable(top)

def main():
    estudios = lee_estudios(r"data\Salud.csv")
    umbral, n = 2, 3
    test_top_enfermedades_mejor_recuperacion_media(estudios, umbral, n)
```



```
umbral = 9.5
test_top_enfermedades_mejor_recuperacion_media(estudios, umbral)
```

```
if __name__=="__main__":
    main()
```

Ejercicio 5

```
def tratamiento_mas_estudiado_por_categoria(estudios: list[Estudio]) \
    -> dict[str, tuple[str, int]]:
    d = agrupar_por_categoria(estudios)
    return {categoria: tratamiento_mas_estudiado(lista_estudios) \
            for categoria, lista_estudios in d.items()}

def agrupar_por_categoria(estudios: list[Estudio]) -> dict[str, list[Estudio]]:
    d = defaultdict(list)
    for e in estudios:
        d[e.categoria].append(e)
    return d

def tratamiento_mas_estudiado(estudios: list[Estudio]) -> tuple[str, int]:
    c = Counter(e.tipo_tratamiento for e in estudios)
    return max(c.items(), key=lambda it: it[1])
    # También return c.most_common(1)[0]

def tratamiento_mas_estudiado2(estudios: list[Estudio]) -> tuple[str, int]:
    #con defaultdict
    c = defaultdict(int)
    for e in estudios:
        c[e.tipo_tratamiento] +=1
    return max(c.items(), key=lambda it: it[1])
```

Test

```
def test_tratamiento_mas_estudiado_por_categoria(estudios: list[Estudio]) -> None:
    dicc_categorias = tratamiento_mas_estudiado_por_categoria(estudios)
    for c, e in dicc_categorias.items():
        print(f"Para la categoría {c}, el tratamiento más estudiado fue {e[0]} con {e[1]} estudios")

def main():
    estudios = lee_estudios(r"data\Salud.csv")
    test_tratamiento_mas_estudiado_por_categoria(estudios)

if __name__=="__main__":
    main()
```